Exercise sheet 3[1]

Coding Exercises

Q3.1 Write a function `swap(...)` that takes two integer parameters and swaps their values, so that the code

```
int a = 5, b = 6;
swap(a, b);
std::cout << a << ", " << b << std::endl;
```

would produce the output

```
6, 5
```

Since the `swap` functions alters its parameters, think about how those parameters must be passed (by value, by reference, or by `const` reference?).

Q3.2 The following function was written to return the maximum and minimum values in a `vector<double>`:

```
1  // Given a vector v, return its maximum and minimum
2  // elements in the variables max_val and min_val, respectively
3  void get_max_min(std::vector<double> v,
4                   double max_val,
5                   double min_val)
6  {
7   max_val = min_val = v[0];
8   unsigned s=v.size();
9   for (unsigned i=0; i<s; i++)
10   {
11    if (v[i] < min_val) min_val = v[i];
12    if (v[i] > max_val) max_val = v[i];
13   }
14 }
```

As written, all three arguments are passed by value. Is this correct? Check for each argument if it should be passed by value, by reference, or by `const` reference. How should the function prototype (lines 3–5) be written?

Also, can you spot another problem with this code? **Hint:** As written, will the code work for vectors of any length?

Q3.3 Write a function that, given a minimum and maximum x_{min} and x_{max} and a number of elements n, returns a `vector<double>` with n equally spaced entries from $x_{\min}$ to $x_{\max}$:

$$x_i = x_{min} + ih, \quad \text{for} \quad i = 0, 1, \ldots, n-1 \quad \text{where} \quad h = \frac{x_{max} - x_{min}}{n-1}$$

[1] Version 28/09/2026.
Feedback, comments, corrections, applause, etc. to: `M.Heil@maths.manchester.ac.uk`

28/09/2026, 07:59:00

(You may recognise this as the `linspace` function from MATLAB).

How would you implement a '`logspace`' function, which takes the same parameters and also returns a vector of n elements ranging from x_{min} to x_{max}, but one in which the *ratio* between each pair of adjacent elements x_i and x_{i+1} is constant?

Q3.4 The mathematics library `<cmath>` has a built-in power function called `pow`. Write a program that uses the built-in power function to evaluate the following expressions:

$$2^3, \quad \left(\frac{4}{9}\right)^{1/2}, \quad (-1)^5, \quad (-64)^{(-1/3)}, \quad 0^0.$$

Can the built-in power function evaluate all these expressions correctly? If not, can you explain why?

Q3.5 Write a program that uses the explicit (forward) Euler's method to find an approximate solution to the initial value problem:

$$\frac{\mathrm{d}x}{\mathrm{d}t} = 1 - x, \qquad x(0) = 0. \tag{1}$$

Reminder: The Euler method for an initial value problem comprising the ODE

$$\mathrm{d}x/\mathrm{d}t = f(x,t)$$

subject to an initial condition

$$x(t=0) = X$$

approximates the time-derivative using a finite-difference approximation with timestep Δt to obtain the approximate solution $x_j \approx x(j\Delta t)$ for $j = 0, 1, 2, \ldots$. The first-order explicit variant of the method uses a forward difference approximation and evaluates the right hand side of the ODE at the current time, yielding

$$\frac{x_{j+1} - x_j}{\Delta t} = f(x_j, j\Delta t).$$

Given the solution at the "current" time, x_j, this expression can be solved for the solution at the next timestep, x_{j+1}, with $x_0 = X$ provided by the initial condition.

Now find the exact solution of the initial value problem (1) and compare your numerical results to the exact solution. How does the error (the difference between the exact and numerical solution) change as time increases? What happens to the error at a fixed time when we reduce the timestep in the numerical integration?

Q3.6 Here is a function that "plots" $\sin(x)$ at n equally-spaced points between a and b (where "plots" means that it outputs n values of x and $\sin(x)$, with the actual plotting then to be done by a separate plotting program).

```
1 // "Plot" sin(x) at n equally spaced points between a and b
```


28/09/2026, 07:59:00

```
2  void plot_it(const unsigned& n, const double &a, const double &b)
3  {
4   // Increment
5   double dx=(b-a)/double(n-1);
6
7   // Loop over plot points
8   double x=a;
9   for (unsigned i=0;i<n;i++)
10   {
11    std::cout << x << " " << sin(x) << std::endl;
12    x+=dx;
13   }
14 }
```

What if we wanted to generalise this function so that it can plot any (user specified) function $f(x)$ rather than just $\sin(x)$? The answer to this is to use **function pointers** which allow us to pass functions to functions. Their syntax is a bit more complicated than that for passing basic types (like doubles or unsigneds): Rather than just passing the type (here a double) and the variable name (here `var`), as in this example

```
1  void some_function(double var)
2  {
3    // Do something with var
4    std::cout << var << std::endl;
5  }
```

we specify (i) the return type of the function, (ii) the name of the function and (iii) (in brackets) the argument(s) to the function. Here's a function, `another_function(...)` that takes a function pointer to function `fct` that takes an unsigned as an argument and returns a double:

```
1  void another_function(double fct(unsigned))
2  {
3    // Evaluate the function specified by fct at i=3
4    unsigned i=3;
5    std::cout << fct(i) << std::endl;
6  }
```

So if we want to be able to "plot" a (mathematical) function of a single argument, $f(x)$, we represent f by a C++ function that returns a double (the value of f evaluated at x) and assume that x is passed to that function as const reference to a double:

```
1  // "Plot" function (which takes a const reference to a double
2  // as an argument and returns a double) specified by function
3  // pointer at n equally spaced points between a and b
4  void general_plot_it(double fct(const double&), const unsigned& n,
5                       const double &a, const double &b)
6  {
7   // Increment
8   double dx=(b-a)/double(n-1);
9
```


28/09/2026, 07:59:00

```
 // Loop over plot points
 double x=a;
 for (unsigned i=0;i<n;i++)
  {
   std::cout << x << " " << fct(x) << std::endl;
   x+=dx;
  }
}
```

If we now specify some function $f(x) = \sin(x) + \cos(x)$ in our code

```
// My function of a single variable
double my_function(const double& x)
{
 return sin(x)+cos(x);
}
```

we can "plot it" by calling

```
 general_plot_it(my_function,n,a,b);
```

Use this approach to write a function `sum(...)` that, given two function pointers to functions of two variables, $f(x, y)$ and $g(x, y)$ evaluates the sum of these functions at specified values of x and y and returns that sum.

Q3.7 Write a function `print_list(...)` that writes the entries in a `std::list` containing `std::string`s to the screen and identifies the "current entry", pointed to by a specified iterator, with an arrow. Here's the start of a `main()` function that illustrates the anticipated use case:

```
// Driver code
int main()
{
 // A list containing strings; lets put in some initial values
 std::list<std::string> my_list = {"one","two","three"};

 // Iterator to the "current" list entry; let's start
 // at the beginning
 std::list<std::string>::iterator it=my_list.begin();

 // Show me the list and what we currently "point at"
 print_list(my_list,it);
```

Q3.8 Extend the `main()` function from the previous question to allow a user to interactively manipulate the list. Assuming the executable is called `linked_list`, here is what should happen when the code is run:

```
> ./linked_list

Current entries in the list:
--> one
```


28/09/2026, 07:59:00

```
    two
    three


Current list entry: one

Now what? Enter:
     "n" to move to and display the next entry (wrap around if you
         reach the end)
     "a" to add an entry before the current one
     "d" to delete the current entry
     "l" to display the entire list and highlight the current entry
     "q" to display the entire list and then quit
```

Once the required action has been performed, the code should return to the options list (until the execution is terminated by entering "q").

Implement these capabilities one by one. Make sure your code handles

- illegal input (e.g. the user enters "bla"); if any unrecognised input is received the code should complain bitterly and then repeat the prompt
- empty lists
- dangling iterators (Note: iterators aren't quite the same as pointers but potential dangliness is a concept that applies to both!).

Note that the deletion of an entry identified by an iterator is different from the deletion of possibly multiple entries based on their value. We've discussed the latter in the lecture; the former uses the member function `erase(...)`. Check the relevant C++ online resources (e.g. `https://en.cppreference.com/w/cpp/container/list/erase.html`) for its use. The function provides some help in dealing with the now dangling iterator (following the deletion, the list entry that the iterator used to point to now no longer exists. The iterator to the "current entry" will therefore have to be updated to point to some other entry). You will have to do a little bit more work to make sure it always points to a valid entry.

28/09/2026, 07:59:00

Analysis exercise

Analysis exercise 3: Numerical Integration
This week's analysis exercise concerns numerical integration by the trapezoidal rule. The trapezoidal rule with n intervals approximates a proper integral of a function f by

$$\int_a^b f(x)\,dx \approx h\left(\frac{1}{2}f(x_0) + f(x_1) + \cdots + f(x_{n-1}) + \frac{1}{2}f(x_n)\right), \tag{2}$$

where $h = (b-a)/n$ is the step size and $x_i = a + h\ i$. The sum converges to the integral as $n \to \infty$.

Tasks:

1. Write a function with prototype `double integrate(double f(double), double a, double b, unsigned n)` that takes a function pointer to the integrand $f(x)$, the limits a and b, and the number of intervals n as parameters. The function should return the value of the integral as calculated by the trapezoidal rule.

   Implement the function in a header file `analysis_exercise3.h`.

2. Write a program `analysis_exercise3.cpp` that uses the header file to evaluate the improper (!) integral

   $$I = \int_0^\infty e^{-x^2}\,\mathrm{d}x \tag{3}$$

   Note that the numerical evaluation of this integral by the trapezoidal rule requires two approximations (and, to obtain the correct result, limits): The accuracy of the approximation (2) for a proper integral (with finite bounds a and b) is controlled by the number of intervals n. Here we also have to choose a sufficiently large value for the upper bound, b, for the proper integral assumed in (2) to be a good approximation to the improper integral (3). Assess how the accuracy of the numerical result depends on both approximations. If you choose a modest number of intervals ($n = 10$, say) does an increase in b improve the result (as you might expect it to!)? Explain what is happening.

   Hint: To assess the accuracy you may use the fact that the exact value of the integral is $\sqrt{\pi}/2$.

3. Check the literature for the theoretical rate at which the computed result is guaranteed to approach the exact value as $n \to \infty$. Compare this theoretical prediction against the behaviour you observe in your computations (Set b to a sufficiently large value so that the numerical result no longer varies when you increase b further; $b = 10$ should do the trick!). Discuss what you find. Is there an error in the analysis or in your code[a]?

Upload the header file, `analysis_exercise3.h` containing the implementation of the trapezoidal rule, and the pdf file containing your results, `analysis_exercise3.pdf` to the course's GRADESCOPE page, accessible via Canvas.

28/09/2026, 07:59:00

It is not necessary to upload the driver code. We will test your header file with our own driver code! Note that for this to work, your header file must not contain the `main()` function!
The pdf file of your report must have **no more than 6 pages**, incl. the title page which must show your student ID.
Our GRADESCOPE-based tests will assess the correctness of the function `integrate(...)`, implemented in `analysis_exercise3.h`. The report should provide your answers to questions 2 and 3.
You can upload your files as many times as you want. The grade for the project will be based on the final version received before the submission deadline:

14:00 on Tuesday of Week 5.

[a] Or in both?

28/09/2026, 07:59:00